Beginner Workshop

Tiny Tapeout — From Transistor to Silicon

Northwestern IEEE · Open Silicon 2026

Date: Friday April 17, 2026 **Room:** Tech L440 **Time:** 5–9 PM

Duration: ≈2 hours of curriculum

Prerequisites: None. Laptop + GitHub account.

All tools are browser-based. No installs required for Levels 1–3.

Contents

1 The Learning Arc **2**

2 What You Need **2**

3 Level 1: Draw a Transistor in SiliWiz **2**

4 Level 2: Design a Logic Gate in Wokwi **3**
4.1 Demo Circuits (Follow Along) . . . 3
4.2 Build Your Own Gate . . . 3

5 Level 3: Generate Your GDS **4**
5.1 Fork the Template . . . 4
5.2 Link Your Wokwi Design . . . 4
5.3 Watch the Action . . . 6
5.4 View in 3D . . . 6

6 From Truth Table to Gates — Automatically **6**

7 Level 4: Verilog with VGA Playground **6**
7.1 What Is This Code? . . . 7
7.2 Goal 1: Reverse the Stripes . . . 7
7.3 Two Kinds of Verilog . . . 7
7.4 Goal 2: Get It on GitHub Actions . . . 8
7.5 Submit . . . 8

8 Level 5: Raw Verilog with OSS CAD Suite **9**
8.1 Install . . . 9
8.2 What Is OSS CAD Suite? . . . 9
8.3 Try It: Simulate and Synthesize . . . 10
8.4 FPGA → ASIC: What to Fix . . . 11
8.5 Check Locally Before Pushing . . . 11
8.6 Get Into Tiny Tapeout . . . 11

9 Key Deadlines **11**

10 Shortcuts Reference **11**

Appendix A: VGA Playground Example Code **13**

Appendix B: Raw Verilog Counter Example **15**

Appendix C: Homework and Further Reading **17**

1 The Learning Arc

How computer science is usually taught:

Arduino → Blinky → Breadboards → Sensors → Libraries → Full projects

We do the same for hardware:

SiliWiz → Logic Gate → Wokwi → VGA Playground → OSS CAD Suite → GDS → Silicon

Hardware step	CS analogy
SiliWiz — draw a transistor	Learning what a resistor is before you plug it into a breadboard
Wokwi — drag-and-drop schematic	Plugging components into a breadboard and testing with LEDs
GitHub Actions — generate GDS	Clicking "Upload" in the Arduino IDE — compile and flash
VGA Playground — Verilog	Writing real C++ instead of using the block editor
OSS CAD Suite + Yosys	Running `gcc` yourself instead of letting the IDE do it
OpenLane — place and route	The linker and assembler that produce the final binary
GDS file	The `.hex` file sent to the programmer
Fabricated chip	Your code running on real hardware

Note

Today: Levels 1–3 in ≈2 hours. Levels 4–5 are post-workshop next steps (see Appendices A–B for example code).

2 What You Need

- Laptop + charger
- GitHub account (`github.com`)
- Chrome or Firefox

No installs. No setup. Everything runs in the browser for Levels 1–3.

3 Level 1: Draw a Transistor in SiliWiz

Link: `tinytapeout.com/guides/workshop/draw-a-mosfet-faster`

Before you plug a resistor into a breadboard, you need to know what a resistor is — what it looks like, how it works, what the bands mean. SiliWiz is the same idea for chip design: before you use transistors, you need to see one, draw one, and understand what it does. Every chip ever made is built from this one component.

A MOSFET is a solid-state switch: a small gate voltage turns on current flow between source and drain. Above the threshold voltage (≈0.5 V) the transistor is on; below it, off. That is a digital 1 and 0.

1. Open SiliWiz. Select `n-diffusion` and draw a horizontal rectangle.
2. Select `polysilicon`. Draw a vertical rectangle crossing the diffusion — the overlap is the gate.
3. Select `metal`. Place contacts at the left end (source), right end (drain), and on top of the poly (gate).
4. Switch to **Cross-section** view — see the physical layer stack.
5. Switch to **Simulation** view — sweep the gate voltage and observe the threshold.

Note

The polysilicon protects the silicon underneath it during ion implantation, so source and drain self-align to the gate edges. No manual alignment needed.

4 Level 2: Design a Logic Gate in Wokwi

Wokwi is your breadboard. You plug in components, wire them together, and test with switches and LEDs — exactly like prototyping a circuit before you solder it. If it works on the breadboard, it will work on the board.

4.1 Demo Circuits (Follow Along)

Open each project and explore it before building your own.

1. **NAND Gate:** `wokwi.com/projects/346663628544410195`
   Toggle inputs 1 and 2 with the DIP switches. Find the combination that turns the red LED on. NAND is LOW only when *both* inputs are HIGH.
2. **D Flip-Flop:** `wokwi.com/projects/346660796151890516`
   When Enable = 1: output follows the data input. When Enable = 0: output holds its last value. This is memory — one bit of it.
3. **Full Adder:** `wokwi.com/projects/346662028519146068`
   Toggle A, B, and Carry-in. Observe Sum and Carry-out. 5 gates to add two bits. Chain 32 of these = a 32-bit adder.

4.2 Build Your Own Gate

Template: `wokwi.com/projects/354858054593504257`

1. Open the template. **Save immediately** (`Ctrl+S`) — this creates your unique copy and project ID. Without saving you cannot submit.
2. Press `+`, add a `74HC00` (Quad NAND chip).
3. Wire: `IN0` → pin 1A, `IN1` → pin 1B, pin 1Y → `OUT0`.

4. Power: VCC → pin 14, GND → pin 7.
5. Click **Play**. Toggle `IN0` and `IN1`. Verify the NAND truth table:

IN0	IN1	OUT0
0	0	1
0	1	1
1	0	1
1	1	0

Note

"Input pin not driven" warning means a chip input is floating. Connect unused inputs to VCC or GND.

5 Level 3: Generate Your GDS

Template: `github.com/TinyTapeout/ttsky-wokwi-template`

Your breadboard prototype works. Now you click "Upload" — the Arduino IDE compiles your sketch, links it, and flashes it to the microcontroller. This step is the same idea: GitHub compiles your schematic into a physical chip layout. You do not need to understand the compiler to use it.

5.1 Fork the Template

1. Go to `github.com/TinyTapeout/ttsky-wokwi-template`.
2. Click **Use this template** → **Create a new repository**. Name it, set to **Public**.

5.2 Link Your Wokwi Design

1. Open `info.yaml` in your repo.
2. Under `project:`, set `wokwi_id` to the number in your saved Wokwi URL: `wokwi.com/projects/XXXXXXXXX`
3. Fill in `title`, `author`, `description`. Commit.

Note

Replace the **entire** contents of `info.yaml` with the example below (swap in your own Wokwi ID, name, and description):

Warning

Do **not** copy-paste from this PDF — invisible characters will break YAML. Download the ready-made file from the workshop repo instead: `github.com/jackthoene/TinyTapeout/blob/master/examples/info.yaml`

`info.yaml` — complete working example

```
project:
  wokwi_id:      460572563557172225
  title:         "Truth Table LED - NU Workshop Demo"
  author:        "Jack Thoene"
```

```
  discord:      ""
  description:  "NAND gate truth table with LED indicators"
  language:     "Wokwi"
  clock_hz:     0
  tiles:        "1x1"
pinout:
  ui[0]: "Switch A"
  ui[1]: "Switch B"
  ui[2]: ""
  ui[3]: ""
  ui[4]: ""
  ui[5]: ""
  ui[6]: ""
  ui[7]: ""
  uo[0]: "NAND output (red LED)"
  uo[1]: "Input A indicator (white LED)"
  uo[2]: "Input B indicator (purple LED)"
  uo[3]: ""
  uo[4]: ""
  uo[5]: ""
  uo[6]: ""
  uo[7]: ""
  uio[0]: ""
  uio[1]: ""
  uio[2]: ""
  uio[3]: ""
  uio[4]: ""
  uio[5]: ""
  uio[6]: ""
  uio[7]: ""
yaml_version: 6
```

Also replace `docs/info.md` so the datasheet page is not blank:

`docs/info.md` — complete working example

```
## How it works

This design implements a 2-input NAND gate truth table
with LED indicators. Two DIP switch inputs (A and B) feed
a NAND gate. The output is displayed on a red LED. Two
additional LEDs mirror the individual input states.

| A | B | NAND Output |
|---|---|-------------|
| 0 | 0 | 1           |
| 0 | 1 | 1           |
| 1 | 0 | 1           |
| 1 | 1 | 0           |

## How to test

1. Toggle ui[0] (Switch A) and ui[1] (Switch B).
2. Observe uo[0] (red LED) for the NAND output.
3. uo[1] and uo[2] show the current input states.
4. Verify all four rows of the truth table above.

## External hardware
```

```
None required. The dev board's built-in DIP switches
and LEDs are sufficient.
```

5.3 Watch the Action

1. Go to the **Actions** tab. Watch `gds` run (yellow → green ✓, or red ×).
2. The action takes ≈5 minutes: fetches schematic → Yosys synthesis → OpenLane place-and-route → DRC → 3D viewer.
3. Wait for both `gds` and `docs` to show green.

Warning

If `gds` fails: wrong `wokwi_id`, design too large for one tile, or a floating port in Wokwi are the most common causes. Check the logs.

5.4 View in 3D

1. **Settings** → **Pages** → **source: GitHub Actions**. Visit your GitHub Pages URL.
2. In the 3D viewer: toggle layers on and off. Find an inverter cell — that is two MOSFETs, drain-to-drain, exactly what you drew in SiliWiz.

Note

The viewer shows the full tile, not just your logic. Most of what you see is routing, tap cells, and antenna diodes added by the toolchain. Your design is the small cluster of cells inside.

6 From Truth Table to Gates — Automatically

Link: tinytapeout.com/digital_design/wokwi_lut_generator/

Remember the NAND truth table? You wired it by hand in Wokwi. But what if you had 4 inputs and 7 outputs — like a BCD-to-7-segment decoder?

The **Wokwi LUT Generator** does it for you:

1. Write a truth table in JSON (inputs → outputs)
2. The tool converts it to boolean equations (Quine-McCluskey minimization)
3. It generates a complete Wokwi schematic — AND gates, OR gates, all wired

This is exactly what Verilog and Yosys do. You describe what the logic should do (the truth table). The tool figures out how to build it (the gates). Verilog is the same idea, but for circuits too complex for a truth table.

7 Level 4: Verilog with VGA Playground

Link: vga-playground.com **Example code:** Appendix A

> *The Arduino block editor is fine for blinking an LED, but real projects need real code. Same here: Wokwi is great for learning, but it caps out at ≈20 gates. Verilog is the C++ of chip design — you describe hardware in text, and the compiler (Yosys) turns it into gates. VGA Playground is your serial monitor: live feedback as you type.*

7.1 What Is This Code?

Open **vga-playground.com** and pick **Stripes**. Here is what each piece does:

Verilog	What it does
`module tt_um_vga_example(...)`	The top-level "box." Inputs/outputs are the 26 pins on your TT tile.
`wire [9:0] pix_x, pix_y;`	Current pixel position (0–639 across, 0–479 down). Generated by `hvsync_generator`.
`reg [9:0] counter;`	A 10-bit register that counts up by 1 each frame (∼60 Hz). This makes things move.
`wire [9:0] moving_x = pix_x + counter;`	Shifts the pixel coordinate right each frame → stripes scroll left.
`assign R = {moving_x[5], pix_y[2]};`	Picks one bit of the shifted position → stripe every 32 pixels. Bit index = stripe width.
`always @(posedge vsync)`	Runs once per frame. Increments `counter`.
`assign uo_out = {hsync, B[0], ...};`	Maps colour and sync signals to the TinyVGA PMOD pin order.
`assign uio_out = 0; assign uio_oe = 0;`	Unused bidirectional pins must be zeroed or CI will fail.

7.2 Goal 1: Reverse the Stripes

Find this line:

```
wire [9:0] moving_x = pix_x + counter;
```

Change `+` to `-`. That is it. One character. The stripes now scroll the other direction.

Why? `+ counter` shifts the pixel coordinate right each frame, so the pattern appears to move left. `- counter` shifts it the other way.

Try next: Change `moving_x[5]` to `[4]` (16 px stripes) or `[6]` (64 px). Change which colours use `moving_x` vs `pix_y`.

> **Note**
> Coloured underlines in the editor mean a syntax error. The screen freezes on the last valid state until you fix it.

7.3 Two Kinds of Verilog

Everything in Verilog is one of two things:

	Combinational	Sequential
Keyword	`assign` / `wire`	`always @(posedge clk)`
When	Every time inputs change	Once per clock edge
Hardware	Pure wires and gates	Flip-flops (memory)
Example	`assign out = a & b;`	`counter <= counter + 1;`
Analogy	A light switch	A latch that remembers

```
// Combinational -- evaluated every time inputs change
assign out = ~(a & b);    // NAND gate

// Sequential -- evaluated on each rising clock edge
always @(posedge clk) begin
    if (!rst_n) count <= 0;     // active-low reset
    else        count <= count + 1;
end
```

Things that simulate but will NOT synthesize:

- `#10` delays — no such thing in real gates
- Division `/` — use `>> 1` instead of `/ 2`
- Large memories — 640×480 B/W requires 38 KB; far too large for one tile

7.4 Goal 2: Get It on GitHub Actions

Once you have stripes moving the way you want, get the GDS synthesizing:

1. Fork **github.com/TinyTapeout/ttihp-verilog-template** (VGA uses the IHP shuttle).
2. **Settings** → **Pages** → **source: GitHub Actions**.
3. Copy your Verilog (or Appendix A) into `src/project.v`.
4. Rename the module: `tt_um_YOURGITHUBNAME_vga`
5. Update `info.yaml`: `title`, `author`, `description`, `top_module`.
6. Update `docs/info.md`: fill in **How it works** and **How to test**.
7. In `test/test.py`, add `cocotb.pass_test()` immediately after the `async def` line.
8. In `test/Makefile`, add `hvsync_generator.v` to `PROJECT_SOURCES`.
9. Commit → wait for `gds` and `docs` green.

Note

`cocotb.pass_test()` skips testing — it tells CI "trust me, it works" so you can get a green checkmark tonight. Writing a real testbench is outside the scope of this workshop. Before you submit for fabrication, you should write proper tests. See: `tinytapeout.com/hdl/testing/`

7.5 Submit

1. Go to **app.tinytapeout.com/projects/create**. Log in with GitHub.
2. **Create a new project**, paste your repo URL.

3. Apply coupon if you have one. Click **Submit a new revision**.
4. To update later: commit again → **Submit a Revision**. Updates accepted until May 9.

8 Level 5: Raw Verilog with OSS CAD Suite

Download: github.com/YosysHQ/oss-cad-suite-build/releases **Example code:** Appendix B

> *When you outgrow the Arduino IDE, you install* `gcc` *and run the toolchain yourself. Same idea: OSS CAD Suite gives you the exact same Yosys that GitHub Actions runs, but locally. You see every step, catch errors before pushing, and understand what the tools actually produce.*

8.1 Install

Download the file for your platform from the releases page:

OS	File to download
Linux (x64)	`oss-cad-suite-linux-x64-*.tgz`
Linux (ARM64)	`oss-cad-suite-linux-arm64-*.tgz`
Mac (Apple Silicon)	`oss-cad-suite-darwin-arm64-*.tgz`
Mac (Intel)	`oss-cad-suite-darwin-x64-*.tgz`
Windows (x64)	`oss-cad-suite-windows-x64-*.exe`

```
# Linux / macOS --- extract and activate
tar -xf oss-cad-suite-*.tgz
source oss-cad-suite/environment

# Windows --- run the .exe installer, then:
oss-cad-suite\environment.bat

# Verify (all platforms)
yosys --version && iverilog -V
```

Note

On macOS you may need to allow the unsigned binary in System Settings → Privacy & Security after the first run attempt.

8.2 What Is OSS CAD Suite?

If you have used Quartus or Synplify, OSS CAD Suite will feel different. There is no GUI, no project wizard, no schematic editor. It is a set of command-line tools you chain together — think `gcc` + `gdb` + `make`, not Visual Studio.

Tool	What it does	Quartus equivalent
`yosys`	RTL synthesis (Verilog → gates)	Analysis & Synthesis
`nextpnr`	Place-and-route (gates → layout)	Fitter
`iverilog`	Verilog simulation	ModelSim / Questa
`gtkwave`	Waveform viewer (VCD files)	SignalTap / ModelSim waves
`sby`	Formal verification (prove properties)	—

Language support:

- **Verilog** — native in Yosys
- **SystemVerilog** — via the `yosys-slang` plugin (IEEE 1800-2017)
- **VHDL** — via the `ghdl-yosys-plugin`

Editing: Use any text editor. VS Code with the *Lushay Code* extension gives syntax highlighting and lint. There is no built-in schematic editor — you write code in a text editor, compile from the terminal, and debug with waveforms.

Debugging: Write a testbench, simulate with `iverilog`, view signals in `gtkwave`. Yosys can also dump a gate-level schematic with `yosys -p "show"` (requires GraphViz).

8.3 Try It: Simulate and Synthesize

Example files are in the workshop repo under `examples/`: `project.v` (4-bit counter) and `tb.v` (testbench). The same code is printed in Appendix B.

```
# Activate the tools (run this every time you open a terminal)
source oss-cad-suite/environment

# cd into the workshop repo (NOT oss-cad-suite, it has its own examples/
    folder)
cd /path/to/workshop-repo

# Simulate: compile and run
iverilog -o sim examples/tb.v examples/project.v
vvp sim

# View waveforms in GTKWave
gtkwave tb.vcd
```

Expected output from simulation:

```
After 20 cycles (expect 4): count = 4
After pause (unchanged): count = 4
After reset (expect 0): count = 0
```

Note

GTKWave tips: In the left panel, expand **tb** → **dut**. Click a signal (e.g. `count[3:0]`, `clk`, `rst_n`, `uo_out`), then click **Append** to add it to the waveform view. Press **Shift+Alt+F** to zoom to fit — without this you will see a blank screen.

```
# Synthesize: same Yosys that GitHub Actions runs
yosys -p "read_verilog examples/project.v; \
          synth -top tt_um_yourname_counter; stat;"

# View gate-level schematic (requires GraphViz)
yosys -p "read_verilog examples/project.v; \
          synth -top tt_um_yourname_counter; \
          show -format png -prefix counter_schematic"
```

Expected output from synthesis: Yosys prints a cell inventory — `_SDFFE_`, `_XOR_`, `_AND_`, etc. These are the primitive gates your counter was compiled into. The `show` command generates a schematic PNG showing how the gates are wired.

8.4 FPGA → ASIC: What to Fix

Issue	FPGA behaviour	ASIC fix
Power-on state	Flops start at 0	Use explicit `rst_n` reset
Reset polarity	Usually active-high	TT uses active-low `rst_n`
`initial` blocks	Work	Ignored in synthesis — remove
Division `/`	Synthesizes (large)	Use `» 1` instead of `/ 2`
`#delay`	Simulation only	Remove — use clock counting

8.5 Check Locally Before Pushing

Before committing to GitHub, run the same checks locally:

```
# Simulate your design + testbench
iverilog -o sim test/tb.v src/project.v && vvp sim

# Check synthesis
yosys -p "read_verilog src/project.v; \
          synth -top tt_um_yourname_counter; stat;"
```

Fix before committing: `multiple drivers` · `undriven port` · `combinational loop`.

8.6 Get Into Tiny Tapeout

1. Fork `github.com/TinyTapeout/ttsky-verilog-template` (sky130) or `github.com/TinyTapeout/ttihp-verilog-template` (IHP).
2. Copy your `.v` file to `src/project.v`. Keep the folder structure as-is.
3. Update `info.yaml` — `top_module` must exactly match the module name.
4. Add `cocotb.pass_test()` in `test/test.py`.
5. Commit → `gds` and `docs` green → app.tinytapeout.com → Submit.

9 Key Deadlines

Date	Milestone
April 17	Beginner workshop
May 1	IEEE form submission
May 5	Token distribution
May 9	**Final submission deadline**

See **Appendix C** for stretch goals and aspirational project examples.

10 Shortcuts Reference

Wokwi

Key	Action
`+`	Add component
`Delete`	Remove selected
`Ctrl+Z`	Undo
`Ctrl+S`	Save
`Space`	Start / stop simulation
Scroll	Zoom
Middle-drag	Pan

SiliWiz

Key	Action
Scroll	Zoom
Click + drag	Draw on selected layer
`Tab`	Switch Layout / Cross-section / Simulation

Appendix A: VGA Playground Example Code

Copy this file to `src/project.v` in your `ttihp-verilog-template` repository. Rename `tt_um_yourname_vga_st` to include your GitHub username. Set `clock_hz: 25000000` in `info.yaml`.

See the **MODIFY THIS SECTION** block to change colours, direction, and stripe width.

Listing 1: VGA Animated Stripes — src/project.v

```
`default_nettype none

// -- VGA sync generator (do not modify)
   ------------------------------------
// Generates hsync, vsync, display_on, hpos[9:0], vpos[9:0]
// for 640x480 @ 60 Hz with a 25 MHz pixel clock.
module hvsync_generator (
    input  wire       clk,
    input  wire       reset,
    output wire       hsync,
    output wire       vsync,
    output wire       display_on,
    output wire [9:0] hpos,
    output wire [9:0] vpos
);
    localparam H_DISPLAY = 640, H_FRONT = 16, H_SYNC = 96, H_BACK = 48;
    localparam V_DISPLAY = 480, V_TOP   = 10, V_SYNC =  2, V_BOTTOM =
       33;
    localparam H_TOTAL   = H_DISPLAY + H_FRONT + H_SYNC + H_BACK;  //
       800
    localparam V_TOTAL   = V_DISPLAY + V_TOP   + V_SYNC + V_BOTTOM; //
       525

    reg [9:0] hcount = 0, vcount = 0;

    always @(posedge clk) begin
        if (reset) begin
            hcount <= 0; vcount <= 0;
        end else if (hcount == H_TOTAL - 1) begin
            hcount <= 0;
            vcount <= (vcount == V_TOTAL - 1) ? 0 : vcount + 1;
        end else
            hcount <= hcount + 1;
    end

    assign hpos       = hcount;
    assign vpos       = vcount;
    assign display_on = (hcount < H_DISPLAY) && (vcount < V_DISPLAY);
    assign hsync      = ~((hcount >= H_DISPLAY + H_FRONT) &&
                          (hcount <  H_DISPLAY + H_FRONT + H_SYNC));
    assign vsync      = ~((vcount >= V_DISPLAY + V_TOP) &&
                          (vcount <  V_DISPLAY + V_TOP + V_SYNC));
endmodule


// -- Main design
   ------------------------------------------------------------
```

```
// Rename module to tt_um_YOURGITHUBNAME_vga_stripes
module tt_um_yourname_vga_stripes (
    input  wire [7:0] ui_in,    // ui_in[0] = optional speed toggle
    output wire [7:0] uo_out,   // VGA: {hsync, B0, G0, R0, vsync, B1,
       G1, R1}
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,      // 25 MHz
    input  wire       rst_n
);
    wire       hsync, vsync, display_on;
    wire [9:0] hpos, vpos;

    hvsync_generator hvsync_gen (
        .clk(clk), .reset(~rst_n),
        .hsync(hsync), .vsync(vsync),
        .display_on(display_on), .hpos(hpos), .vpos(vpos)
    );

    // Frame counter: increments once per frame (~60 Hz)
    reg [9:0] frame_count;
    reg       vsync_r;
    always @(posedge clk) begin
        if (!rst_n) begin
            frame_count <= 0; vsync_r <= 1;
        end else begin
            vsync_r <= vsync;
            if (vsync_r && !vsync)   // falling edge = new frame
                frame_count <= frame_count + 1;
        end
    end

    // -- MODIFY THIS SECTION
       ----------------------------------------------------
    // Change + to - to reverse stripe direction.
    // Change [5] to [4] for 16 px stripes, [6] for 64 px stripes.
    // Try different r/g/b combinations for different colours.
    wire [9:0] moving_x = hpos + frame_count;
    wire stripe = moving_x[5];           // stripe every 32 pixels

    wire r = display_on &  stripe;       // red on stripe
    wire g = 1'b0;
    wire b = display_on & ~stripe;       // blue off stripe
    //
       ----------------------------------------------------------------------


    // TT VGA pinout: {hsync, B0, G0, R0, vsync, B1, G1, R1}
    assign uo_out  = {hsync, b, g, r, vsync, b, g, r};
    assign uio_out = 8'b0;
    assign uio_oe  = 8'b0;

endmodule
```

Appendix B: Raw Verilog Counter Example

Two files are provided: the design (`project.v`) and a simulation testbench (`tb.v`). Both are in the `examples/` directory of the workshop repo, ready to run. To submit: copy `project.v` to `src/project.v` in your `ttsky-verilog-template` repository and rename the module to `tt_um_YOURGITHUBNAME_counter`.

Commands (with OSS CAD Suite active)

Listing 2: Simulate and check synthesis locally

```
# Activate tools (every new terminal)
source oss-cad-suite/environment

# cd into the workshop repo (NOT oss-cad-suite, it has its own examples/
    folder)
cd /path/to/workshop-repo

# Simulate
iverilog -o sim examples/tb.v examples/project.v
vvp sim

# View waveforms (expand tb > dut, append signals,
# then Shift+Alt+F to zoom to fit)
gtkwave tb.vcd

# Synthesize + view gate-level schematic
yosys -p "read_verilog examples/project.v; \
          synth -top tt_um_yourname_counter; stat; show"
```

Warnings to fix before committing: `multiple drivers` · `undriven port` · `combinational loop`.

Listing 3: 4-bit counter — src/project.v

```
`default_nettype none

module tt_um_yourname_counter (
    input  wire [7:0] ui_in,    // ui_in[0]: hold HIGH to pause counting
    output wire [7:0] uo_out,   // uo_out[3:0]: count value on lower 4
        outputs
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,
    input  wire       rst_n     // active-low reset (ASIC convention)
);
    reg [3:0] count;

    always @(posedge clk) begin
        if (!rst_n)               count <= 4'b0;
        else if (ena & ~ui_in[0]) count <= count + 1;
    end

    // All unused outputs must be explicitly assigned 0
```

```
    assign uo_out  = {4'b0, count};
    assign uio_out = 8'b0;
    assign uio_oe  = 8'b0;

endmodule
```

Listing 4: Simulation testbench — test/tb.v

```
`default_nettype none
`timescale 1ns/1ps

module tb;
    reg        clk    = 0;
    reg        rst_n  = 0;
    reg        ena    = 1;
    reg  [7:0] ui_in  = 8'b0;
    reg  [7:0] uio_in = 8'b0;
    wire [7:0] uo_out, uio_out, uio_oe;

    tt_um_yourname_counter dut (
        .ui_in(ui_in), .uo_out(uo_out),
        .uio_in(uio_in), .uio_out(uio_out), .uio_oe(uio_oe),
        .ena(ena), .clk(clk), .rst_n(rst_n)
    );

    always #5 clk = ~clk;   // 100 MHz clock (10 ns period)

    initial begin
        $dumpfile("tb.vcd");
        $dumpvars(0, tb);
        #20 rst_n = 1;        // release reset after 2 cycles

        #200;
        $display("After 20 cycles (expect 4): count = %0d", uo_out[3:0])
            ;

        ui_in[0] = 1;         // pause
        #100;
        $display("After pause (unchanged): count = %0d", uo_out[3:0]);
        ui_in[0] = 0;

        #50 rst_n = 0;        // mid-run reset
        #20 rst_n = 1;
        $display("After reset (expect 0): count = %0d", uo_out[3:0]);

        #100 $finish;
    end
endmodule
```

Appendix C: Homework and Further Reading

Things to try between the workshop (April 17) and the final submission deadline (May 9). Submit your tile at `app.tinytapeout.com/projects/create`. Tokens will be distributed by May 5.

Level 1 Stretch: CMOS Inverter in SiliWiz

Open `app.siliwiz.com` and build the inverter from a blank canvas before revealing the example. Place NMOS first, then PMOS above it in an n-well. Connect their gates together as the input and their drains together as the output. This is the standard cell that Yosys inserts into every synthesized design.

Level 2 Stretch: More Wokwi Designs

Tutorials: `tinytapeout.com/digital_design/`

Half adder
Add `74HC86` (XOR) and `74HC08` (AND). Wire: IN0 XOR IN1 → OUT0 (sum), IN0 AND IN1 → OUT1 (carry). This is the innermost arithmetic building block of every CPU.

AND + OR comparison
Add `74HC08` (AND) and `74HC32` (OR). Wire: IN0 AND IN1 → OUT0, IN0 OR IN1 → OUT1. The outputs differ when exactly one input is high — that gap is XOR, the basis of addition.

Clock divider
Add a Clock Generator and a chain of `74HC74` D flip-flops. Each flip-flop divides the clock frequency by 2. Watch the LEDs blink at 10 kHz → 5 kHz → 2.5 kHz...

7-segment decoder
Tutorial: `tinytapeout.com/digital_design/7segment/` Drive a 7-segment display from a 4-bit BCD input.

Combination lock
Tutorial: `tinytapeout.com/digital_design/padlock/` Build a state machine that unlocks an output only on the correct 4-input sequence.

Level 4 Aspirational: Advanced VGA Projects

TT Logo Screensaver
`github.com/TinyTapeout/tt-logo-screensaver`
Bouncing logo with colour gradients and gamepad support. Officially maintained, well-commented. Good "next step" after the basic stripes example.

Demoscene competition entries
Browse previous entries at `vga-playground.com`. VGA Mandelbrot, VGA Nyan Cat, VGA Perlin Noise, VGA Pong with NES Controllers — all fit in one tile.

Level 5 Aspirational: Bigger Verilog Designs

Ben Eater's 8-bit CPU
`github.com/jasonkaufmann/tt07-beneater8bit` — SystemVerilog
Ports the famous YouTube breadboard CPU to a single TT tile. Accumulator, B register,

ALU, 16-byte RAM, program counter, 16 instructions. If you have watched his videos you already understand the architecture.

Simon Says memory game
github.com/urish/tt07-simon-game — Verilog + cocotb testbench
Small enough to read in an afternoon. Has a real CI testbench to learn from.

DFFRAM 32×32
github.com/TinyTapeout/tt05-dffram-example — 99.7% Verilog
1024-bit single-port RAM built from flip-flops. Shows how memory is built without SRAM macros.

Spiking neural network
github.com/rejunity/tt05-spiking-neural-net — Verilog
40 neurons, 320 synapses, binarized leaky integrate-and-fire in under 10K gates on $\approx 1\,\mathrm{mm}^2$.

Where to Find More Examples

- All TT08 projects (96 designs): tinytapeout.github.io/tinytapeout-08/shuttle_index.json
- All TT07 projects (120+ designs): tinytapeout.github.io/tinytapeout-07/shuttle_index.json
- Curated list: github.com/mattvenn/awesome-opensource-asic-resources
- Full course (not free): zerotoasiccourse.com